

EASWARI ENGINEERING COLLEGE

(Autonomous)

RAMAPURAM, CHENNAI-89

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



LAB MANUAL

REGULATION R23 VERSION 1.1

2311CSC302J – DATA STRUCTURES AND ALGORITHMS

ACADEMIC YEAR: 2025-2026

B.E – CSE

II YEAR / III SEMESTER

Prepared By

Approved By

V. VISHNUGANDHI, Assistant Professor/CSE(CS)

HOD/CSE(CS)

COURSE OBJECTIVES

1. To impart the basic concepts of data structures and algorithms
2. To understand basic concepts about stacks and queues.
3. To introduce various techniques for representation of the data in the real world.
4. To develop application using data structures.
5. To understand concepts about searching and sorting techniques

COURSE OUTCOMES

Course Name: 2311CSC302J – Data Structures and Algorithms		Year / Sem: II / III
Upon completion of the course, the students will be able to:		
CO1	Infer the concepts of abstract data types for linear data structures.	
CO2	Apply non-linear data structures concepts in real time applications.	
CO3	Develop solutions using linear and non- linear data structures.	
CO4	Illustrate various searching and sorting techniques to solve the complex problem.	
CO5	Evaluate space and time complexity for a given algorithm.	

MAPPING OF COURSE OUTCOMES WITH THE PROGRAM OUTCOMES

CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
C01	1	2	2	1	-	-	-	-	-	2	1	1
C02	1	2	1	1	-	-	-	-	-	1	-	1
C03	2	2	1	1	-	-	-	-	-	1	-	1
C04	2	2	1	1	-	-	-	-	-	1	-	1
C05	1	3	2	2	-	2	1	-	-	1	1	1

MAPPING OF COURSE OUTCOMES WITH THE PROGRAM SPECIFIC OUTCOMES

CO	PSO1	PSO2	PSO3
C01	2	2	2
C02	2	2	2
C03	2	2	2
C04	2	2	2
C05	2	2	2

LIST OF EXPERIMENTS

1. Array implementation of List ADT
2. Linked list implementation of Stack ADT
3. Linked list implementation of Queue ADT
4. Implementation of Breadth First Search Traversal
5. Implementation of Binary Search
6. Implement Bubble sort using C

TOTAL: 30 PERIODS

TABLE OF CONTENTS

Ex. No.	Date	Title of the Experiment	Page No.	Mark	Signature
1		Array implementation of List ADT			
2		Linked List implementation of Stack ADT			
3		Linked List implementation of Queue ADT			
4		Implementation of Breadth First Search Traversal			
5		Implementation of Binary Search			
6		Implement Bubble Sort using C			
CONTENT BEYOND SYLLABUS					
7		Implementation of Linear Search			
8		Implementation of Binary Search Trees and Traversal			
9		Implement Quick Sort using C			
10		Implementation of Hashing - any one Collision Resolution Techniques			

Ex NO: 1	ARRAY IMPLEMENTATION OF LIST ADT
DATE:	

AIM:

To implement the List ADT using arrays in C, supporting the following operations:

- Insertion
- Deletion
- Display
- Search

ALGORITHM:

Initialization

1. **Create** an integer array list[MAX] to hold the elements.
2. **Initialize** an integer variable size = 0 to keep track of the number of elements in the list.

1. Insert Operation

Input: Position pos, Value val

Steps:

1. If size == MAX, print "Overflow" and exit.
2. If pos < 0 or pos > size, print "Invalid position" and exit.
3. Shift all elements from pos to size - 1 one position to the right:
 - for i = size to pos+1 (in reverse)
 - list[i] = list[i - 1]
4. Set list[pos] = val.
5. Increment size by 1.
6. Print "Inserted successfully".

2. Delete Operation

Input: Position pos

Steps:

1. If size == 0, print "Underflow" and exit.
2. If pos < 0 or pos >= size, print "Invalid position" and exit.
3. Shift all elements from pos + 1 to size - 1 one position to the left:

- for i = pos to size - 2
list[i] = list[i + 1]
- 4. Decrement size by 1.
- 5. Print "Deleted successfully".

3. Display Operation

Steps:

1. If size == 0, print "List is empty" and exit.
2. Loop from i = 0 to size - 1:
 - Print list[i]

4. Search Operation

Input: Value key

Steps:

1. Loop from i = 0 to size - 1:
 - If list[i] == key, return position i.
2. If loop completes without finding key, return -1.

5. Exit Operation

- End the program when the user chooses to exit.

PROGRAM

```
#include <stdio.h>
#define MAX 100

int list[MAX];
int size = 0;

void insert(int pos, int value);
void delete(int pos);
void display();
int search(int key);

int main() {
    int choice, pos, value, key;

    while (1) {
        printf("\nList Operations:\n");
```

```

printf("1. Insert\n2. Delete\n3. Display\n4. Search\n5. Exit\n");
printf("Enter choice: ");
scanf("%d", &choice);

switch (choice) {
    case 1:
        printf("Enter position (0 to %d): ", size);
        scanf("%d", &pos);
        printf("Enter value: ");
        scanf("%d", &value);
        insert(pos, value);
        break;

    case 2:
        printf("Enter position to delete (0 to %d): ", size - 1);
        scanf("%d", &pos);
        delete(pos);
        break;

    case 3:
        display();
        break;

    case 4:
        printf("Enter value to search: ");
        scanf("%d", &key);
        pos = search(key);
        if (pos != -1)
            printf("Element found at position %d\n", pos);
        else
            printf("Element not found\n");
        break;

    case 5:
        return 0;

    default:
        printf("Invalid choice!\n");
}
}
return 0;
}

void insert(int pos, int value) {
    if (size == MAX) {
        printf("List is full (overflow).\n");
        return;
    }

```

```

    }
    if (pos < 0 || pos > size) {
        printf("Invalid position.\n");
        return;
    }

    for (int i = size; i > pos; i--)
        list[i] = list[i - 1];

    list[pos] = value;
    size++;
    printf("Inserted successfully.\n");
}

void delete(int pos) {
    if (size == 0) {
        printf("List is empty (underflow).\n");
        return;
    }
    if (pos < 0 || pos >= size) {
        printf("Invalid position.\n");
        return;
    }

    for (int i = pos; i < size - 1; i++)
        list[i] = list[i + 1];

    size--;
    printf("Deleted successfully.\n");
}

void display() {
    if (size == 0) {
        printf("List is empty.\n");
        return;
    }

    printf("List elements: ");
    for (int i = 0; i < size; i++)
        printf("%d ", list[i]);
    printf("\n");
}

int search(int key) {
    for (int i = 0; i < size; i++) {
        if (list[i] == key)
            return i;
    }
}

```



```
    }  
    return -1;  
}
```

SAMPLE OUTPUT

List Operations:

1. Insert
2. Delete
3. Display
4. Search
5. Exit

Enter choice: 1

Enter position (0 to 0): 0

Enter value: 50

Inserted successfully.

Enter choice: 1

Enter position (0 to 1): 1

Enter value: 70

Inserted successfully.

Enter choice: 3

List elements: 50 70

Enter choice: 4

Enter value to search: 70

Element found at position 1

Enter choice: 2

Enter position to delete (0 to 1): 0

Deleted successfully.

Enter choice: 3

List elements: 70

RESULT

The List ADT in C was successfully implemented using a simple array. The program supports:

- Insertion at any position
- Deletion from any position
- Searching for a value
- Displaying the list

Ex NO: 2	LINKED LIST IMPLEMENTATION OF STACK ADT
DATE:	

AIM

To implement the **Stack Abstract Data Type (ADT)** using a **singly linked list** in C, supporting:

- push() – insert at top
- pop() – delete from top
- peek() – return top element
- display() – show all elements

ALGORITHM

Let each node contain data and a pointer next.

Initialization:

1. Define a Node with fields: data and next.
2. Initialize top = NULL.

PUSH Operation:

Input: Value to insert

1. Create a new node.
2. Assign data = value.
3. Set new_node->next = top.
4. Set top = new_node.

POP Operation:

Output: Value deleted

1. If top == NULL, print "Underflow" and exit.
2. Store top->data in a variable.
3. Move top = top->next.
4. Free the removed node.
5. Return the stored value.

PEEK Operation:

1. If top == NULL, print "Stack is empty".
2. Else, print top->data.

DISPLAY Operation:

1. If top == NULL, print "Stack is empty".
2. Traverse from top to NULL, print each data.

C PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

// Define node structure
typedef struct Node {
    int data;
    struct Node* next;
} Node;

// Stack top pointer
Node* top = NULL;

// Function prototypes
void push(int value);
void pop();
void peek();
void display();

int main() {
    int choice, value;

    while (1) {
        printf("\nStack Operations:\n");
        printf("1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;
```

```

        case 2:
            pop();
            break;

        case 3:
            peek();
            break;

        case 4:
            display();
            break;

        case 5:
            exit(0);

        default:
            printf("Invalid choice!\n");
    }
}

return 0;
}

// Push function
void push(int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Heap Overflow\n");
        return;
    }
    newNode->data = value;
    newNode->next = top;
    top = newNode;
    printf("Pushed %d to stack\n", value);
}

// Pop function
void pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }
    Node* temp = top;

```

```

    printf("Popped %d from stack\n", top->data);
    top = top->next;
    free(temp);
}

```

// Peek function

```

void peek() {
    if (top == NULL) {
        printf("Stack is empty\n");
        return;
    }
    printf("Top element is %d\n", top->data);
}

```

// Display function

```

void display() {
    if (top == NULL) {
        printf("Stack is empty\n");
        return;
    }
    Node* current = top;
    printf("Stack elements: ");
    while (current != NULL) {
        printf("%d ", current->data);
        current = current->next;
    }
    printf("\n");
}

```

SAMPLE OUTPUT

Stack Operations:

1. Push

2. Pop

3. Peek

4. Display

5. Exit

Enter choice: 1

Enter value to push: 10

Pushed 10 to stack

Enter choice: 1

Enter value to push: 20

Pushed 20 to stack

Enter choice: 3
Top element is 20

Enter choice: 4
Stack elements: 20 10

Enter choice: 2
Popped 20 from stack

Enter choice: 4
Stack elements: 10

RESULT

The Stack ADT with all operations (push, pop, peek, and display) was successfully implemented using a linked list in C.

Ex NO: 3	LINKED LIST IMPLEMENTATION OF QUEUE ADT
DATE:	

AIM

To implement the Queue Abstract Data Type (ADT) using a singly linked list **in C**, supporting:

- enqueue() – insert at rear
- dequeue() – delete from front
- peek() – return front element
- display() – show all elements

ALGORITHM

Let each node contain:

- data: the element
- next: pointer to the next node

Use two pointers:

- front: points to the first node
- rear: points to the last node

ENQUEUE (Insert at Rear)

Input: Value to insert

1. Create a new node with data = value and next = NULL
2. If rear == NULL, set both front and rear to the new node
3. Else, set rear->next = new node, then update rear = new node

DEQUEUE (Remove from Front)

Output: Value removed

1. If front == NULL, queue is empty → underflow
2. Store front node in temp, retrieve its data
3. Move front = front->next
4. If front == NULL, set rear = NULL (queue became empty)
5. Free the removed node

PEEK (Front Element)

1. If front == NULL, queue is empty

2. Else, return front->data

DISPLAY

1. Traverse from front to NULL, printing each data

C PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

// Define node structure
typedef struct Node {
    int data;
    struct Node* next;
} Node;

// Front and rear pointers
Node* front = NULL;
Node* rear = NULL;

// Function prototypes
void enqueue(int value);
void dequeue();
void peek();
void display();

int main() {
    int choice, value;

    while (1) {
        printf("\nQueue Operations:\n");
        printf("1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;

            case 2:
                dequeue();
```



```

        break;

    case 3:
        peek();
        break;

    case 4:
        display();
        break;

    case 5:
        exit(0);

    default:
        printf("Invalid choice!\n");
    }
}

return 0;
}

```

```

// Enqueue function
void enqueue(int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (!newNode) {
        printf("Memory allocation failed\n");
        return;
    }
    newNode->data = value;
    newNode->next = NULL;

    if (rear == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
    printf("Enqueued %d\n", value);
}

```

```

// Dequeue function
void dequeue() {
    if (front == NULL) {
        printf("Queue Underflow (empty)\n");
    }
}

```

```

        return;
    }
    Node* temp = front;
    printf("Dequeued %d\n", front->data);
    front = front->next;
    if (front == NULL)
        rear = NULL;
    free(temp);
}

// Peek function
void peek() {
    if (front == NULL) {
        printf("Queue is empty\n");
    } else {
        printf("Front element is %d\n", front->data);
    }
}

// Display function
void display() {
    if (front == NULL) {
        printf("Queue is empty\n");
        return;
    }
    Node* current = front;
    printf("Queue elements: ");
    while (current != NULL) {
        printf("%d ", current->data);
        current = current->next;
    }
    printf("\n");
}

```

SAMPLE OUTPUT

Queue Operations:

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit

Enter your choice: 1

Enter value to enqueue: 10

Enqueued 10

Enter your choice: 1

Enter value to enqueue: 20

Enqueued 20

Enter your choice: 4

Queue elements: 10 20

Enter your choice: 2

Dequeued 10

Enter your choice: 3

Front element is 20

Enter your choice: 4

Queue elements: 20

RESULT

The Queue ADT was successfully implemented using a linked list in C.

All operations – enqueue, dequeue, peek, and display – work correctly and handle empty queue cases.

Ex NO: 4	BREADTH FIRST SEARCH (BFS) TRAVERSAL ON A GRAPH
DATE:	

AIM

To implement Breadth First Search (BFS) traversal on a graph using C.

ALGORITHM

BFS(G, start_vertex):

1. Initialize a visited[] array to 0.
2. Create a **queue** and enqueue the start_vertex.
3. Mark start_vertex as **visited**.
4. While the queue is not empty:
 - Dequeue a vertex v
 - Visit v (print or store it)
 - For each adjacent vertex u of v:
 - If u is not visited:
 - Mark u as visited
 - Enqueue u

C PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int adj[MAX][MAX]; // Adjacency matrix
int visited[MAX]; // Visited array
int queue[MAX]; // Queue
int front = -1, rear = -1;

// Queue operations
void enqueue(int value) {
    if (rear == MAX - 1) return;
    if (front == -1) front = 0;
    queue[++rear] = value;
}

int dequeue() {
    if (front == -1 || front > rear) return -1;
    return queue[front++];
}
```

```

int isEmpty() {
    return (front == -1 || front > rear);
}

// BFS traversal
void bfs(int n, int start) {
    for (int i = 0; i < n; i++) visited[i] = 0;

    enqueue(start);
    visited[start] = 1;

    printf("BFS Traversal starting from vertex %d: ", start);

    while (!isEmpty()) {
        int v = dequeue();
        printf("%d ", v);

        for (int i = 0; i < n; i++) {
            if (adj[v][i] == 1 && !visited[i]) {
                enqueue(i);
                visited[i] = 1;
            }
        }
    }
    printf("\n");
}

// Main function
int main() {
    int n, e, u, v, start;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter number of edges: ");
    scanf("%d", &e);

    // Initialize adjacency matrix to 0
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            adj[i][j] = 0;

    // Read edges

```

```

printf("Enter edges (u v):\n");
for (int i = 0; i < e; i++) {
    scanf("%d %d", &u, &v);
    adj[u][v] = 1;
    adj[v][u] = 1; // Comment this line if graph is directed
}

printf("Enter starting vertex: ");
scanf("%d", &start);

bfs(n, start);

return 0;
}

```

SAMPLE OUTPUT

Enter number of vertices: 5

Enter number of edges: 6

Enter edges (u v):

0 1

0 2

1 3

1 4

2 4

3 4

Enter starting vertex: 0

BFS Traversal starting from vertex 0: 0 1 2 3 4

RESULT

The Breadth First Search (BFS) algorithm was successfully implemented using an adjacency matrix in C, and the traversal output is generated from a given starting node.

Ex NO: 5	IMPLEMENTATION OF BINARY SEARCH
DATE:	

AIM

To implement the Binary Search algorithm in C for searching an element in a sorted array.

ALGORITHM

Input: Sorted array arr[], number of elements n, key to search key

Steps:

1. Set low = 0 and high = n - 1
2. While low <= high:
 - Compute mid = (low + high) / 2
 - If arr[mid] == key, return mid
 - If arr[mid] < key, set low = mid + 1
 - Else set high = mid - 1
3. If not found, return -1

C PROGRAM

```
#include <stdio.h>
```

```
int binarySearch(int arr[], int n, int key) {
```

```
    int low = 0, high = n - 1, mid;
```

```
    while (low <= high) {
```

```
        mid = (low + high) / 2;
```

```
        if (arr[mid] == key)
```

```
            return mid;
```

```
        else if (arr[mid] < key)
```

```
            low = mid + 1;
```

```
        else
```

```
            high = mid - 1;
```

```
    }
```

```
    return -1; // Not found
```

```
}
```

```
int main() {  
    int arr[100], n, key, i, result;  
  
    printf("Enter number of elements: ");  
    scanf("%d", &n);  
  
    printf("Enter %d sorted elements:\n", n);  
    for (i = 0; i < n; i++)  
        scanf("%d", &arr[i]);  
  
    printf("Enter element to search: ");  
    scanf("%d", &key);  
  
    result = binarySearch(arr, n, key);  
  
    if (result == -1)  
        printf("Element not found in the array.\n");  
    else  
        printf("Element found at index %d.\n", result);  
    return 0;  
}
```

SAMPLE OUTPUT

Enter number of elements: 5

Enter 5 sorted elements:

1 3 5 7 9

Enter element to search: 5

Element found at index 2.

RESULT

The Binary Search algorithm was successfully implemented in C.

Ex NO: 6	IMPLEMENT BUBBLE SORT USING C
DATE:	

AIM

To implement the **Bubble Sort algorithm** in C to sort an array in **ascending order**.

ALGORITHM

Input: Array arr[] of n elements

Output: Sorted array in ascending order

Steps:

1. Repeat n-1 times:
 - For i = 0 to n - pass - 1:
 - If arr[i] > arr[i+1], swap them
2. Stop when the array is fully sorted

Optional: Use a swapped flag to optimize by stopping early if no swaps are made.

C PROGRAM

```
#include <stdio.h>
```

```
void bubbleSort(int arr[], int n) {
```

```
    int i, j, temp;
```

```
    int swapped;
```

```
    for (i = 0; i < n - 1; i++) {
```

```
        swapped = 0; // optimization
```

```
        for (j = 0; j < n - i - 1; j++) {
```

```
            if (arr[j] > arr[j + 1]) {
```

```
                // swap arr[j] and arr[j+1]
```

```
                temp = arr[j];
```

```
                arr[j] = arr[j + 1];
```

```
                arr[j + 1] = temp;
```

```

        swapped = 1;
    }
}

// If no two elements were swapped in inner loop, stop
if (!swapped)
    break;
}
}

```

```

void printArray(int arr[], int n) {
    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

```

```

int main() {
    int arr[100], n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    bubbleSort(arr, n);
    printArray(arr, n);

    return 0;
}

```

SAMPLE OUTPUT

Enter number of

elements: 5

Enter 5

elements:

64 25 12 22 11

Sorted array: 11 12 22 25 64

RESULT

The Bubble Sort algorithm was successfully implemented in C.

Ex NO: 7	IMPLEMENTATION OF LINEAR SEARCH
DATE:	

Aim

To write a 'C' program to implement Linear Search.

Algorithm

Step 1: Initialize the integer variables

Step 2: Get the number of elements and its values

Step 3: Start from the leftmost element of arr[] and one by one compare x with each element of arr[]

Step 4: If x matches with an element, return the index.

Step 5: If x doesn't match with any of elements, return -1.

Step 6: Stop the program

Program

```
#include<stdio.h>
int main()
{
    int a[20],i,x,n;
    printf("How many elements?:");
    scanf("%d",&n);

    printf("Enter array elements:n");
    for(i=0;i<n;++i)
        scanf("%d",&a[i]);

    printf("\nEnter element to search:");
    scanf("%d",&x);

    for(i=0;i<n;++i)
        if(a[i]==x)
            break;

    if(i<n)
        printf("Element found at index %d",i);
    else
        printf("Element not found");

    return 0;
}
```

Output:

```
How many elements?10
Enter array elements:
21
22
34
12
66
77
87
99
09
39

Enter element to search:87
Element found at index 6

...Program finished with exit code 0
Press ENTER to exit console. □
```

Result

Thus the 'C' program to implement the Linear search has been executed and the output is verified successfully.

Ex NO: 8	IMPLEMENTATION OF BINARY SEARCH TREES AND TRAVERSALS
DATE:	

Aim

To write a 'C' program to implement Binary Search Tree .

Algorithm

Step 1: Start
Step 2: Create a Binary Search Tree for N elements.
Step 3: Traverse the tree in inorder.
Step 4: Traverse the tree in preorder
Step 6: Traverse the tree in postorder.
Step 7: Search the given key element in the BST.
Step 8: Delete an element from BST.
Step 9: Stop

Program

```
#include <stdlib.h>
typedef struct tnode
{
    int data;
    struct tnode *right,*left;
}TNODE;

TNODE *CreateBST(TNODE *, int);
void Inorder(TNODE *);
void Preorder(TNODE *);
void Postorder(TNODE *);
main()
{
    TNODE *root=NULL;          /* Main Program */
    int opn,elem,n,i;
    do
    {
        clrscr();
        printf("\n ### Binary Search Tree Operations ### \n\n");
        printf("\n Press 1-Creation of BST");
        printf("\n    2-Traversal in Inorder");
        printf("\n    3-Traversal in Preorder");
        printf("\n    4-Traversal in Postorder");
        printf("\n    5-Exit\n");
        printf("\n    Your option ? ");
        scanf("%d",&opn);
        switch(opn)
        {
            case 1: root=NULL;
                    printf("\n\nBST for How Many Nodes ?");
                    scanf("%d",&n);
```

```

        for(i=1;i<=n;i++)
        {
            printf("\nRead the Data for Node %d ?",i);
            scanf("%d",&elem);
            root=CreateBST(root,elem);
        }
        printf("\nBST with %d nodes is ready to Use!!\n",n);
        break;
    case 2: printf("\n BST Traversal in INORDER \n");
            Inorder(root); break;
    case 3: printf("\n BST Traversal in PREORDER \n");
            Preorder(root); break;
    case 4: printf("\n BST Traversal in POSTORDER \n");
            Postorder(root); break;
    case 5: printf("\n\n Terminating \n\n"); break;
    default: printf("\n\nInvalid Option !!! Try Again !! \n\n");
            break;
        }
        printf("\n\n\n Press a Key to Continue . . . ");
        getch();
    }while(opn != 5);
}
TNODE *CreateBST(TNODE *root, int elem)
{
    if(root == NULL)
    {
        root=(TNODE *)malloc(sizeof(TNODE));
        root->left= root->right = NULL;
        root->data=elem;
        return root;
    }
    else
    {
        if( elem < root->data )
            root->left=CreateBST(root->left,elem);
        else
            if( elem > root->data )
                root->right=CreateBST(root->right,elem);
            else
                printf(" Duplicate Element !! Not Allowed !!!");
        return(root);
    }
}
void Inorder(TNODE *root)
{
    if( root != NULL)
    {
        Inorder(root->left);
        printf(" %d ",root->data);
        Inorder(root->right);    } }

void Preorder(TNODE *root)
{

```

```

    if( root != NULL)
    {
        printf(" %d ",root->data);
        Preorder(root->left);
        Preorder(root->right);
    }
}
void Postorder(TNODE *root)
{
    if( root != NULL)
    {
        Postorder(root->left);
        Postorder(root->right);
        printf(" %d ",root->data);
    }
}

```

Output

OPERATIONS ---

- 1 - Insert an element into tree
- 2 - Delete an element from the tree
- 3 - Inorder Traversal
- 4 - Preorder Traversal
- 5 - Postorder Traversal
- 6 - Exit

Enter your choice : 1

Enter data of node to be inserted : 40

Enter your choice : 1

Enter data of node to be inserted : 20

Enter your choice : 1

Enter data of node to be inserted : 10

Enter your choice : 1

Enter data of node to be inserted : 30

Enter your choice : 1

Enter data of node to be inserted : 60

Enter your choice : 3

10 -> 20 -> 30 -> 40 -> 60 -> 80 -> 90 ->

Result

Thus, the 'C' program to implement Binary search Tree has been executed and the output is verified successfully.

Ex NO: 9	IMPLEMENTATION OF QUICK SORT
DATE:	

Aim

To write a 'C' program to implement Quick sort.

Algorithm

Step 1: Choose the highest index value has pivot

Step 2: Take two variables to point left and right of the list excluding pivot

Step 3: left points to the low index

Step 4: right points to the high

Step 5: while value at left is less than pivot move right

Step 6: while value at right is greater than pivot move left

Step 7: If both step 5 and step 6 does not match swap left and right

Step 8: If $\text{left} \geq \text{right}$, the point where they met is new pivot

Program

```
#include<stdio.h>
void quicksort(int number[25],int first,int last)
{
int i, j, pivot, temp;
if(first<last)
{
pivot=first;
i=first;
j=last;
while(i<j)
{
while(number[i]<=number[pivot]&& i<last)
i++;
while(number[j]>number[pivot])
j--;
if(i<j)
{
temp=number[i];
number[i]=number[j];
number[j]=temp;
}
}
temp=number[pivot];
```

```
number[pivot]=number[j];
number[j]=temp;
quicksort(number,first,j-1);
quicksort(number,j+1,last);
}
}
int main()
{
int i, count, number[25];
printf("Enter some elements (Max. - 25): ");
scanf("%d",&count);
printf("Enter %d elements: ", count);
for(i=0;i<count;i++)
scanf("%d",&number[i]);
quicksort(number,0,count-1);
printf("The Sorted Order is: ");
for(i=0;i<count;i++)
printf(" %d",number[i]);
return 0;
}
```

Output

Enter some elements (Max. - 25): 5

Enter 5 elements: 5 22 -19 63 1

The Sorted Order is: -19 1 5 22 63

Result

Thus the 'C' program to implement Quick sort has been executed and the output is verified successfully.

Ex NO: 10	HASHING – ANY TWO COLLISION TECHNIQUES
DATE:	

Aim

To write a C program to implement Hashing using Linear and Quadratic Probing.

Algorithm

Step 1: Create a structure, data (hash table item) with key and value as data.

Step 2: Now create an array of structure, data of some certain size. But, the size of array must be immediately updated to a prime number just greater than initial array

Step 3: A menu is displayed on the screen.

Step 4: User must choose one option from four choices given in the menu

Step 5: Perform all the operations

Step 6: Stop the program

Program

```
#include <stdio.h>
#include <conio.h>
int tsize;
int hasht(int key)
{
    int i ;
    i = key%tsize ;
    return i;
}
//-----LINEAR PROBING-----
int rehashl(int key)
{
    int i ;
    i = (key+1)%tsize ;
    return i ;
}
//-----QUADRATIC PROBING-----
int rehashq(int key, int j)
{
    int i ;
    i = (key+(j*j))%tsize ;
    return i ; }
void main()
{
    int key,arr[20],hash[20],i,n,s,op,j,k ;
    clrscr() ;
    printf ("Enter the size of the hash table: ");
    scanf ("%d",&tsize);
    printf ("\nEnter the number of elements: ");
```

```

    scanf ("%d",&n);
    for (i=0;i<tsize;i++)
hash[i]=-1 ;
    printf ("Enter Elements: ");
    for (i=0;i<n;i++)
    {
scanf("%d",&arr[i]);
    }
    do
    {
printf("\n\n1.Linear Probing\n2.Quadratic Probing \n3.Exit \nEnter your option: ");
scanf("%d",&op);
switch(op)
{
case 1:
for (i=0;i<tsize;i++)
hash[i]=-1 ;
for(k=0;k<n;k++)
{
key=arr[k] ;
i = hasht(key);
while (hash[i]!=-1)
{
i = rehashl(i);
}
hash[i]=key ;
}
printf("\nThe elements in the array are: ");
for (i=0;i<tsize;i++)
{
printf("\n Element at position %d: %d",i,hash[i]);
}
break ;
case 2:
for (i=0;i<tsize;i++)
hash[i]=-1 ;
for(k=0;k<n;k++)
{
j=1;
key=arr[k] ;
i = hasht(key);
while (hash[i]!=-1)
{
i = rehashq(i,j);
j++ ;
}
hash[i]=key ;
}
printf("\nThe elements in the array are: ");
for (i=0;i<tsize;i++)
{
printf("\n Element at position %d: %d",i,hash[i]);
}

```

```
    break ;  
}  
}while(op!=3);  
getch() ; }
```

Output

Enter the size of the hash table: 10

Enter the number of elements: 8

Enter Elements: 72 27 36 24 63 81 92 101

1.Linear Probing
2.Quadratic Probing
3.Exit
Enter your option: 1

The elements in the array are:

Element at position 0: -1
Element at position 1: 81
Element at position 2: 72
Element at position 3: 63
Element at position 4: 24
Element at position 5: 92
Element at position 6: 36
Element at position 7: 27
Element at position 8: 101
Element at position 9: -1

1.Linear Probing
2.Quadratic Probing
3.Exit
Enter your option: 2

Result

Thus the 'C' program to implement Hashing using Linear and Quadratic Probing has been executed and the output is verified successfully.

#####